Page Replacement Algorithm Simulator – Project Report

Course: Operating Systems | **Topic:** Virtual Memory / Page Replacement

Name: _____ **Roll No.:** _____ **Date:** 25 October 2026

Language: Python (Jupyter Notebook) | **Source file:** `Page_Replacement_Simulator.ipynb` | **GitHub:** _____

1. Introduction

In a demand-paged system, a process runs with only some of its pages in main memory. When it references a page that is not in a frame, a **page fault** occurs. If every frame is full, the operating system must pick a **victim page** to replace. A reference to a page already in memory is a **page hit**. The quality of the replacement policy strongly affects performance, because each fault costs a disk access.

This project implements a simulator for three replacement algorithms: **FIFO**, **LRU** and **Optimal**. The user enters a reference string and a number of frames at run time. The program runs all three algorithms on the same input, prints a step-by-step trace for each, and finishes with a comparison of page faults, hits, hit ratio and fault ratio.

Objectives

- Explain how demand paging triggers page faults and why page replacement is needed.
- Implement FIFO, LRU and Optimal replacement.
- Compare the algorithms using faults, hits, hit ratio and fault ratio.
- Observe Belady's anomaly experimentally.

2. Algorithms

Algorithm	Victim selection rule	Needs	Time per run (n refs, k frames)
FIFO	The page that has been in memory the longest	A queue of arrival order	O(n·k)
LRU	The page not used for the longest time in the past	Recency of every access	O(n·k)

Algorithm	Victim selection rule	Needs	Time per run (n refs, k frames)
Optimal	The page used farthest in the future, or never again	Knowledge of future references	$O(n^2 \cdot k)$ worst case

2.1 FIFO (First-In, First-Out)

FIFO keeps a queue of pages in the order they entered memory. When a fault occurs and no frame is free, the page at the front of the queue (the oldest) is replaced and the new page joins the back of the queue. It is simple and cheap, but it ignores how often or how recently a page is used, so a heavily used page can be evicted just because it was loaded early. FIFO also suffers from **Belady's anomaly**.

2.2 LRU (Least Recently Used)

LRU assumes that the recent past predicts the near future. It records the time of last use of each page, on both hits and faults, and evicts the page with the oldest last-use time. It performs close to Optimal on programs with locality of reference, but a real system needs hardware support or extra bookkeeping to track recency.

2.3 Optimal (OPT / MIN)

Optimal looks ahead in the reference string and evicts the page whose next use is farthest away, or which is never used again. It gives the **minimum possible number of faults**, but it cannot be built in a real OS because future references are unknown. It is used only as a benchmark. **Tie rule used in this project:** if several pages are never used again, the one that appears first in the frames is replaced.

3. Implementation

The program is a single Jupyter notebook that uses `pandas` to display the comparison table. All frames start empty and fill in slot order. A victim is replaced in the same slot, so frame contents in each trace row are shown in stable slot order.

Function / cell	Purpose
`parse_reference_string(text)`	Splits on commas/spaces and converts to integers. Raises a `ValueError` for empty, non-integer or negative pages.

Function / cell	Purpose
Interactive input cell	Uses `input()` to read the reference string and the frame count. Converts the frame count with `int()` and rejects values below 1 with a clear error message.
`simulate(reference, num_frames, algorithm)`	Runs FIFO, LRU or Optimal and returns `(faults, hits, trace)`. Also checks that `num_frames` is at least 1.
`print_trace(reference, num_frames, algorithm)`	Prints the table (step, page, frame contents, H/F, replaced page) and the totals: faults, hits, hit ratio, fault ratio.
`compare_algorithms(reference, num_frames)`	Runs all three algorithms on the same input, prints their traces and a final `pandas` comparison table.

A single function, `simulate()`, implements all three policies; only the victim-selection rule differs. FIFO takes the front of a queue of arrival order, LRU takes the resident page with the smallest last-used index, and Optimal takes the resident page whose next use is farthest away (ties go to the first page in the frames). Each trace row is a dictionary with the keys `step`, `page`, `frames`, `result` (H or F) and `replaced` (the evicted page, or `-`). Hit ratio is `hits / n` and fault ratio is `faults / n`, where `n` is the length of the reference string.

Input validation examples

Input	Message
`1 2 x 4`	Page references must be whole numbers separated by commas or spaces.
(empty)	Enter at least one page number.
`1, -2, 3`	Page numbers must be non-negative integers.
frames `abc` or `2.5`	Number of frames must be a whole number.
frames `0` or `-3`	Number of frames must be at least 1.

The complete source code is in the submitted notebook `Page_Replacement_Simulator.ipynb`.

4. Sample Input and Output

Input: reference string `1 2 3 1 4 2 5`, frames = `3`

Algorithm	Faults	Hits	Hit ratio	Fault ratio
FIFO	5	2	28.57%	71.43%
LRU	6	1	14.29%	85.71%
Optimal	5	2	28.57%	71.43%

Step-by-step trace for Optimal on this input:

Step	Page	Frames	H/F	Replaced
1	1	[1]	F	-
2	2	[1, 2]	F	-
3	3	[1, 2, 3]	F	-
4	1	[1, 2, 3]	H	-
5	4	[4, 2, 3]	F	1
6	2	[4, 2, 3]	H	-
7	5	[5, 2, 3]	F	4

Execution screenshots (insert from your Jupyter run):

- Screenshot 1: user entering the input and the FIFO trace table.
- Screenshot 2: LRU and Optimal trace tables.
- Screenshot 3: the final comparison table.
- Screenshot 4: the input validation error messages.

5. Part B – Manual Tracing

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 | **Frames:** 3 (F = fault, H = hit; a dash means the frame is empty)

FIFO

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2	-	0	0	0	0	3	3	3	2	2	2	2	2
Frame 3	-	-	1	1	1	1	0	0	0	3	3	3	3
F / H	F	F	F	F	H	F	F	F	F	F	F	H	H

LRU

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2	-	0	0	0	0	0	0	0	0	3	3	3	3
Frame 3	-	-	1	1	1	3	3	3	2	2	2	2	2
F / H	F	F	F	F	H	F	H	F	F	F	F	H	H

Optimal

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2
Frame 2	-	0	0	0	0	0	0	4	4	4	0	0	0
Frame 3	-	-	1	1	1	3	3	3	3	3	3	3	3
F / H	F	F	F	F	H	F	H	F	H	H	F	H	H

Summary

Algorithm	Page faults	Page hits	Hit ratio
FIFO	10	3	23.08%
LRU	9	4	30.77%
Optimal	7	6	46.15%

The hand-traced results were verified against the program output.

6. Part C – Experiments and Analysis

6.1 Experiment 1: Belady's anomaly

Reference string: 1 2 3 4 1 2 5 1 2 3 4 5

Algorithm	Faults (3 frames)	Faults (4 frames)
FIFO	9	10
LRU	10	8
Optimal	7	6

Q1. Does any algorithm produce more faults with more frames? Yes. **FIFO** goes from 9 faults with 3 frames to 10 faults with 4 frames. This is Belady's anomaly.

Q2. Why do LRU and Optimal never show this? They are **stack algorithms**. For any reference string, the set of pages in memory with *n* frames is always a subset of the set in memory with *n+1* frames (the inclusion or stack property). So an extra frame can only keep more useful pages and can never cause an extra fault. FIFO chooses victims by arrival order rather than usefulness, so it has no such property.

6.2 Experiment 2: Own input (4 frames)

Case	Reference string	Frames	FIFO faults	LRU faults	Optimal faults
LRU better	1 1 1 4 3 5 5 4 3 2 2 1 4 2 4 3 2 (17 pages)	4	8	6	5
FIFO better/equal	5 3 5 2 3 1 1 2 3 5 5 2 1 3 2 3 4 1 (18 pages)	4	5	5	5

LRU better. At step 12, page 1 faults and all four frames are full. FIFO evicts page 4 because it is the oldest, yet page 4 is needed again at steps 13 and 15, so FIFO faults again on 4 and later on 3. LRU evicts page 5, which was last used at step 7 and is never needed again, so the later references to 2, 4 and 3 are all hits. Result: 8 faults for FIFO against 6 for LRU. **FIFO equal.** This string has only 5 distinct pages, so with 4 frames just one eviction is ever needed (step 17, when page 4 first appears). FIFO and LRU both evict page 5, so all three algorithms give the minimum of 5 faults. This case is a tie rather than a FIFO win, which the assignment allows ("better than or equal to"). Overall, LRU beats FIFO when recently used pages are reused soon, and results depend on the access pattern.

6.3 Short questions

1. Why can Optimal not be implemented in a real OS? It needs to know future page references, which depend on program input and execution and cannot be known in advance. It is used only as a benchmark to judge other algorithms.

2. What extra hardware or bookkeeping does LRU need compared with FIFO? LRU must record the recency of every memory access. This needs a hardware counter or timestamp per page, or a stack or linked list of pages updated on each reference. Systems often use approximations such as reference bits, aging or the Clock algorithm. FIFO only needs a queue updated when a page is loaded.

3. If frames equal the number of distinct pages, how many faults will every algorithm produce? Exactly as many faults as there are distinct pages. Only the first reference to each page faults. Since no page is ever evicted, all later references are hits and the replacement policy makes no difference.

7. Comparison of Results

Test input	Frames	FIFO faults	LRU faults	Optimal faults	Best
Sample: 1 2 3 1 4 2 5	3	5	6	5	FIFO and Optimal (tie)
Part B string (13 refs)	3	10	9	7	Optimal
Belady string (12 refs)	3	9	10	7	Optimal
Belady string (12 refs)	4	10	8	6	Optimal
Own string, LRU case (17 refs)	4	8	6	5	Optimal
Own string, FIFO/equal case (18 refs)	4	5	5	5	All three (tie)

Observations

1. **Optimal never has more faults than the others.** It is the lower bound and shows how much room for improvement the practical algorithms have.
2. **LRU is usually better than FIFO** on inputs with locality (Part B, the 4-frame Belady run, the own LRU case), because it keeps recently used pages.
3. **LRU is not always better.** On the sample input FIFO beat LRU (5 against 6), on the 3-frame Belady string FIFO beat LRU (9 against 10), and on the own equal case they tied.
4. **FIFO can get worse with more memory** (Belady's anomaly: 9 to 10 faults). LRU and Optimal improve or stay equal as frames increase.

5. **Cost versus benefit.** FIFO is the cheapest to implement, LRU needs recency tracking, and Optimal is impossible in practice. Real systems therefore use LRU approximations such as Clock.

8. Conclusion

The simulator implements FIFO, LRU and Optimal page replacement with validated user input, step-by-step traces and a comparison table. The experiments confirmed that Optimal gives the fewest faults, that LRU generally beats FIFO but not on every input, and that FIFO exhibits Belady's anomaly while the stack algorithms LRU and Optimal do not. The project helped in understanding how the choice of victim page affects the number of page faults and why practical systems approximate LRU.

9. References

1. A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, Virtual Memory chapter (page replacement algorithms, Belady's anomaly).
2. Course lab assignment: *Page Replacement Algorithm Simulator*, Operating Systems Lab.